

Condicionais, Funções e Funções de Alta Ordem em Python

Construindo Abstrações com Dados e Procedimentos

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação – DCOMP

Expressões Condicionais e Predicados

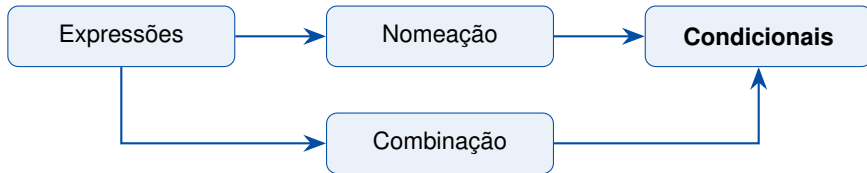
Funções: Meios de Abstração

Funções de Alta Ordem

Expressões Condicionais e Predicados

Ideia central

“O poder expressivo de uma linguagem é severamente limitado sem alguma forma de teste que permita guiar o fluxo de execução.”



Os **três pilares** da programação: expressões primitivas, meios de combinação e meios de abstração. Condicionais são o primeiro salto de complexidade.

Tipo

Python possui dois valores booleanos:

- `True` — verdadeiro
- `False` — falso

```
1 >>> type(True)
2 <class 'bool'>
3 >>> 3 > 2
4 True
5 >>> 3 < 2
6 False
```

Operadores de comparação

Op.	Significado
<code>==</code>	Igual
<code>!=</code>	Diferente
<code><</code>	Menor que
<code>></code>	Maior que
<code><=</code>	Menor ou igual
<code>>=</code>	Maior ou igual

Atenção

Não confunda = (atribuição) com `==` (comparação de igualdade).

Conectivos lógicos em Python

Op.	Significado	Exemplo
<code>and</code>	Conjunção	<code>x > 0 and x < 10</code>
<code>or</code>	Disjunção	<code>x < 0 or x > 10</code>
<code>not</code>	Negação	<code>not(x > 0)</code>

Predicado: função que retorna `bool`

```
def eh_par(n):  
    return n % 2 == 0
```

Avaliação em curto-circuito

Python **não** avalia o segundo operando se o resultado já puder ser determinado:

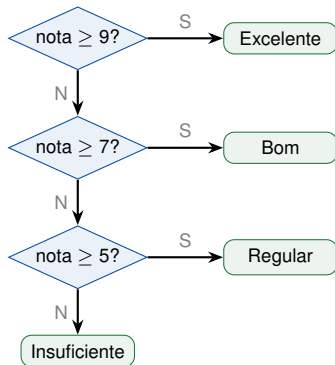
```
# Se x == 0, o segundo  
# operando nao eh avaliado  
x != 0 and 10 / x > 1
```

`and`: para no primeiro `False`
`or`: para no primeiro `True`

Análise de caso

Em linguagens funcionais, usa-se `cond.` Em Python, usamos:

```
1 def classifica_nota(nota):
2     if nota >= 9.0:
3         return "Excelente"
4     elif nota >= 7.0:
5         return "Bom"
6     elif nota >= 5.0:
7         return "Regular"
8     else:
9         return "Insuficiente"
```



Forma compacta do

Quando queremos uma expressão (não um bloco de comandos):

```
1 >>> x = 10
2 >>> "par" if x % 2 == 0 else "impar"
3 'par'
```

`<valor_se_verdadeiro> if <predicado> else <valor_se_falso>`

Analogia funcional

Corresponde à forma especial (if <predicado> <consequente> <alternativa>) de linguagens funcionais como Scheme/Lisp.

Em Python, todo valor tem uma “verdade” associada

Falsy (avaliados como `False`)

- `False`, `None`
- Zero numérico: `0`, `0.0`, `0j`
- Sequências vazias: `""`, `[]`, `()`, `{}`

Truthy (avaliados como `True`)

- Qualquer número diferente de zero
- Qualquer sequência não vazia
- Qualquer objeto (por padrão)

```
1 lista = [1, 2, 3]
2 if lista:           # truthy: lista não está vazia
3     print("Tem elementos!")
```

Problema

Implemente a função `valor_absoluto(x)` usando condicionais:

$$|x| = \begin{cases} x & \text{se } x > 0 \\ 0 & \text{se } x = 0 \\ -x & \text{se } x < 0 \end{cases}$$

Problema

Implemente a função `valor_absoluto(x)` usando condicionais:

$$|x| = \begin{cases} x & \text{se } x > 0 \\ 0 & \text{se } x = 0 \\ -x & \text{se } x < 0 \end{cases}$$

```
1 def valor_absoluto(x):  
2     if x > 0:  
3         return x  
4     elif x == 0:  
5         return 0  
6     else:  
7         return -x
```

Problema

Implemente a função `valor_absoluto(x)` usando condicionais:

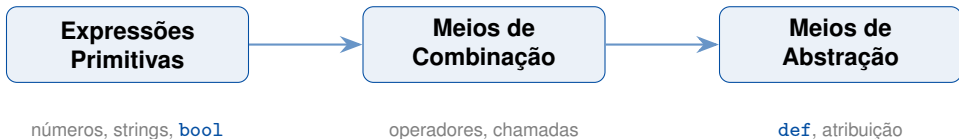
$$|x| = \begin{cases} x & \text{se } x > 0 \\ 0 & \text{se } x = 0 \\ -x & \text{se } x < 0 \end{cases}$$

```
1 def valor_absoluto(x):  
2     if x > 0:  
3         return x  
4     elif x == 0:  
5         return 0  
6     else:  
7         return -x
```

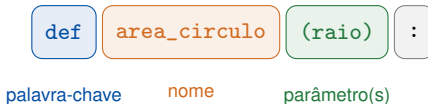
Funções: Meios de Abstração

A ideia central

*“Funções são o principal meio de abstração em qualquer linguagem de programação. Elas permitem associar um **nome** a um padrão computacional composto.”*



```
1 def area_circulo(raio):  
2     """Calcula a area de um circulo dado o raio."""  
3     pi = 3.14159265  
4     return pi * raio ** 2
```

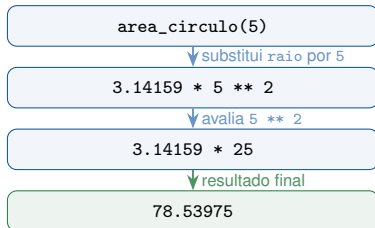


corpo

```
# docstring + corpo  
return pi * raio ** 2
```

Como Python avalia uma chamada de função?

1. Avalia o operador (identifica a função).
2. Avalia os operandos (argumentos).
3. Substitui os parâmetros pelos argumentos no corpo.
4. Avalia o corpo da função.



Encapsulamento

Parâmetros e variáveis definidos dentro de uma função são **locais** — não afetam o ambiente externo.

```
1 def quadrado(x):  
2     return x * x  
3  
4 def soma_quadrados(a, b):  
5     return quadrado(a) + quadrado  
6         (b)  
7  
8 >>> soma_quadrados(3, 4)  
9 25
```

Ambiente Global

quadrado → func
soma_quadrados → func

Local: soma_quadrados

a → 3
b → 4

Local: quadrado

x → 3 (depois 4)

O poder da composição

Funções simples podem ser **compostas** para resolver problemas complexos.

```
1 def quadrado(x):
2     return x * x
3
4 def soma_quadrados(a, b):
5     return quadrado(a) + quadrado(b)
6
7 def f(a):
8     return soma_quadrados(a + 1, a * 2)
9
10 >>> f(5)
11 # soma_quadrados(6, 10) -> quadrado(6) + quadrado(10)
12 # -> 36 + 100 -> 136
13 136
```

Princípio

Cada função realiza **uma tarefa bem definida**. A complexidade emerge da composição, não de funções gigantes.



O **usuário** da função não precisa saber *como* ela foi implementada.

Ele só precisa saber *o que* ela faz (contrato/interface).

Isso é chamado de **abstração procedimental**.

Função Pura ✓

```
def dobro(x):  
    return x * 2
```

```
# Sempre retorna o mesmo  
# resultado para a mesma  
# entrada. Sem efeitos  
# colaterais.
```

Função Impura ✗

```
contador = 0  
def incrementa():  
    global contador  
    contador += 1  
    return contador
```

```
# O resultado depende de  
# estado externo.
```

Boa prática

Prefira **funções puras**: são mais fáceis de testar, depurar e compor. Efeitos colaterais devem ser isolados e controlados.

Problema

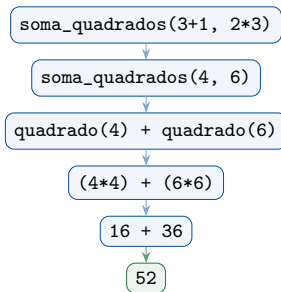
Usando o modelo de substituição, avalie a expressão:

```
soma_quadrados(3 + 1, 2 * 3)
```

Problema

Usando o modelo de substituição, avalie a expressão:

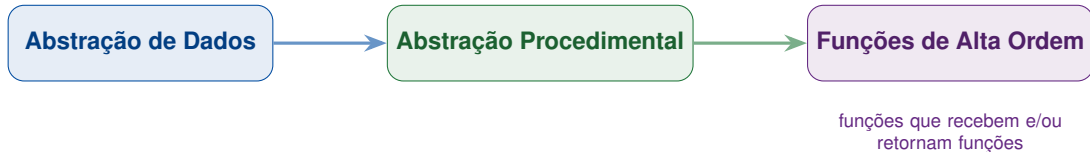
```
soma_quadrados(3 + 1, 2 * 3)
```



Funções de Alta Ordem

A grande ideia

“Funções que manipulam funções são chamadas funções de alta ordem. Elas são um mecanismo de abstração poderoso.”



Uma função de alta ordem é uma função que:

- recebe uma ou mais funções como **argumento**, e/ou
- **retorna** uma função como resultado.

Padrão repetido: somatório

Observe a repetição de padrão nestas funções:

```
1  # Soma de 1 a n
2  def soma_naturais(n):
3      total = 0
4      for i in range(1, n+1):
5          total += i
6      return total
7
8  # Soma dos quadrados
9  def soma_quadrados(n):
10     total = 0
11     for i in range(1, n+1):
12         total += i * i
```

Problema

O **padrão** é o mesmo: apenas a *transformação* de cada termo muda.

$$\sum_{i=1}^n f(i)$$

Podemos abstrair f !

```
1 def somatorio(n, funcao):
2     """Soma funcao(i) para i de 1 ate n."""
3     total = 0
4     for i in range(1, n + 1):
5         total += funcao(i)
6     return total
```

```
1 def identidade(x):
2     return x
3
4 def quadrado(x):
5     return x * x
6
7 def cubo(x):
8     return x * x * x
```

```
>>> somatorio(5, identidade) # 1+2+3+4+5
15
>>> somatorio(5, quadrado) # 1+4+9+16+25
55
>>> somatorio(5, cubo) # 1+8+27+64+125
225
```

Definição

Uma expressão `lambda` cria uma função **sem nome**, útil para funções simples e pontuais.

```
1 # Equivalentes:
2 def quadrado(x):
3     return x * x
4
5 quadrado = lambda x: x * x
```

Uso com funções de alta ordem

```
1 >>> somatorio(5, lambda x: x * x)
2 55
3 >>> somatorio(5, lambda x: x ** 3)
4 225
5 >>> somatorio(10, lambda x: 1/x)    # serie harmonica
6 2.9289682539682538
```

`lambda` parâmetros: expressão

Apenas uma expressão, sem `return` explícito.

Fábricas de funções

Uma função pode **construir e retornar** outra função.

```
1 def cria_somador(n):
2     """Retorna uma funcao que soma n ao argumento."""
3     def somador(x):
4         return x + n
5     return somador
6
7 >>> soma5 = cria_somador(5)
8 >>> soma5(10)
9 15
10 >>> soma5(20)
11 25
12 >>> cria_somador(100)(1)
13 101
```

Closure (fechamento)

A função interna `somador` “lembra” o valor de `n` do ambiente em que foi criada. Isso é chamado de **closure**.

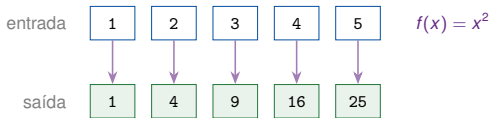
```
1 def compor(f, g):
2     """Retorna a composicao f(g(x))."""
3     return lambda x: f(g(x))
4
5 def dobro(x):
6     return x * 2
7
8 def incrementa(x):
9     return x + 1
10
11 >>> dobro_depois_incrementa = compor(dobro, incrementa)
12 >>> dobro_depois_incrementa(5)
13 12 # dobro(incrementa(5)) = dobro(6) = 12
14
15 >>> incrementa_depois_dobro = compor(incrementa, dobro)
16 >>> incrementa_depois_dobro(5)
17 11 # incrementa(dobro(5)) = incrementa(10) = 11
```

Observe

A **ordem** importa: $f \circ g \neq g \circ f$ em geral.

Aplicar uma função a cada item de uma sequência

```
1 >>> numeros = [1, 2, 3, 4, 5]
2 >>> list(map(lambda x: x ** 2, numeros))
3 [1, 4, 9, 16, 25]
4
5 >>> nomes = ["ana", "bob", "carlos"]
6 >>> list(map(str.upper, nomes))
7 ['ANA', 'BOB', 'CARLOS']
8
9 >>> list(map(len, nomes))
10 [3, 3, 6]
```



Manter apenas os itens que satisfazem um predicado

```
1 >>> numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 >>> list(filter(lambda x: x % 2 == 0, numeros))
3 [2, 4, 6, 8, 10]
4
5 >>> palavras = ["gato", "sol", "computador", "py", "ufs"]
6 >>> list(filter(lambda s: len(s) > 3, palavras))
7 ['gato', 'computador']
```

é par?

1

2

3

4

5

6

7

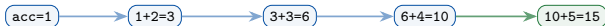
8

9

10

Combinar todos os elementos em um único resultado

```
1 from functools import reduce
2
3 >>> numeros = [1, 2, 3, 4, 5]
4 >>> reduce(lambda acc, x: acc + x, numeros)
5 15
6
7 >>> reduce(lambda acc, x: acc * x, numeros)
8 120 # fatorial de 5
```



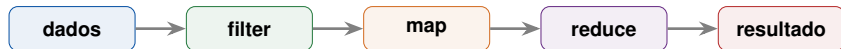
Padrão geral

`reduce(f, [a, b, c, d])` $\Rightarrow$ `f(f(f(a, b), c), d)`

Problema

Dado uma lista de números, calcule a soma dos quadrados dos números pares.

```
1 from functools import reduce
2
3 numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4
5 # Pipeline funcional
6 pares = filter(lambda x: x % 2 == 0, numeros)
7 quadrados = map(lambda x: x ** 2, pares)
8 resultado = reduce(lambda a, b: a + b, quadrados)
9
10 print(resultado) # 4+16+36+64+100 = 220
```



Decoradores são funções de alta ordem

Um **decorador** em Python é uma função que recebe uma função e retorna uma versão "melhorada" dela.

```
1 def debug(func):
2     def wrapper(*args):
3         print(f"Chamando {func.__name__}{args}")
4         resultado = func(*args)
5         print(f"  -> {resultado}")
6         return resultado
7     return wrapper
8
9 @debug
10 def fatorial(n):
11     if n <= 1:
12         return 1
13     return n * fatorial(n - 1)
14
15 >>> fatorial(4)
16 # Chamando fatorial(4,)
17 # Chamando fatorial(3,)
18 # Chamando fatorial(2,)
19 # Chamando fatorial(1,)
20 #   -> 1
21 #   -> 2
22 #   -> 6
23 #   -> 24
```

Conceito	Scheme/Lisp	Python
Condicional	<code>cond, if</code>	<code>if/elif/else</code>
Predicado	<code>(> x 0)</code>	<code>x > 0</code>
Definição de função	<code>(define (f x) ...)</code>	<code>def f(x): ...</code>
Função anônima	<code>(lambda (x) ...)</code>	<code>lambda x: ...</code>
HOF como argumento	<code>(sum f a b)</code>	<code>somatorio(n, f)</code>
HOF retorna função	<code>retorna lambda</code>	<code>return lambda ...</code>
Aplicar a todos	<code>map</code>	<code>map(f, lista)</code>
Filtrar	<code>filter</code>	<code>filter(pred, lista)</code>
Acumular	<code>accumulate</code>	<code>reduce(f, lista)</code>

Desafio

Implemente a função `soma_geral` que generaliza o somatório:

$$\sum_{i=a}^b f(i) \quad \text{onde } f \text{ e os limites são parâmetros}$$

Desafio

Implemente a função `soma_geral` que generaliza o somatório:

$$\sum_{i=a}^b f(i) \quad \text{onde } f \text{ e os limites são parâmetros}$$

```
1 def soma_geral(f, a, b):  
2     """Soma f(i) para i de a ate b."""  
3     total = 0  
4     i = a  
5     while i <= b:  
6         total += f(i)  
7         i += 1  
8     return total
```